

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – COMPARATIVE ANALYSIS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Comparative Analysis
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	RAGAVI S	Reg. No.:	192565027
Section / Batch:	SLOT – A (2025-29)	Date:	24-08-2026

Instructions to Students

- Identify the relevant concepts of L1, L2, and L3 cache, SRAM, DRAM, NAND Flash, MRAM, RRAM, virtual memory, paging, latency, bandwidth, and throughput.
- Analyze the memory requirements of different computer systems by considering cache levels, memory technologies, access latency, bandwidth, throughput, capacity, and energy consumption.
- Evaluate the performance characteristics of different memory technologies and determine their suitability for cache memory, main memory, secondary storage, and future hierarchical memory systems.
- Compute relevant performance measures such as access time, latency, bandwidth, throughput, hit/miss rate, average memory access time (AMAT), speedup, and energy efficiency wherever applicable.
- Interpret the results by assessing memory bottlenecks, performance improvements, technology trade-offs, and the suitability of each memory technology or memory hierarchy for a given application.

QUESTIONS

1. Analyze the relationship between cache size, cache access time, and cache hit rate. Evaluate how changes in cache size affect processor performance, memory latency, and overall system throughput.
2. Assess the impact of memory bandwidth on data-intensive applications such as artificial intelligence, scientific computing, and multimedia processing. Analyze how insufficient bandwidth can create memory bottlenecks and limit processor performance.
3. Evaluate the effect of page size on virtual memory performance. Analyze how page size influences page faults, memory utilization, translation overhead, and system throughput.
4. Examine the role of data locality in improving cache performance. Analyze how temporal and spatial locality influence cache hit rates, memory access latency, and processor efficiency.
5. Assess the suitability of SRAM, DRAM, NAND Flash, MRAM, and RRAM for different levels of a memory hierarchy. Evaluate each technology based on latency, bandwidth, capacity, energy consumption, endurance, and cost, and recommend appropriate technologies for specific applications.

Code of Conduct

I RAGAVI S / 192565027 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: RAGAVI S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Q. No	Scope of Items (20)	Comparison Depth (25)	Use of Evidence (20)	Critical Thinking (20)	Clarity of Presentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Question 1

1. Relationship Between Cache Size, Access Time, and Hit Rate

Cache size and hit rate: As cache size increases, more recently and frequently used data blocks can be retained on-chip, so the probability that a requested address is already present (a "hit") rises. This follows directly from temporal and spatial locality — a larger cache captures a wider working set of the program before eviction, reducing capacity misses.

Cache size and access time: A larger cache is not free — it requires more SRAM cells, longer bit-lines and word-lines, larger decoders, and often deeper set-associativity. Physically, this increases wire delay, capacitance, and the depth of the tag-comparison logic, so access time (cycle time to read/write the cache) increases roughly with the square root to logarithm of capacity, depending on organization.

The trade-off curve: Hit rate improves with diminishing returns as size grows (following a curve similar to a power law), while access time grows monotonically. Beyond a certain size, the marginal improvement in hit rate no longer compensates for the added latency per access — this is why L1 caches are kept small and fast (few KB, 1–4 cycles) while L2/L3 are larger and slower (hundreds of KB to tens of MB, tens of cycles).

2. Effect on Processor Performance, Latency, and Throughput

The combined effect is best captured using the Average Memory Access Time (AMAT) formula:

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

- Increasing cache size lowers Miss Rate, which reduces AMAT even though Hit Time rises slightly — net effect is usually positive up to the point of diminishing returns.
- Beyond the optimal size, the increase in Hit Time outweighs the marginal drop in Miss Rate, and AMAT begins to rise again — performance degrades despite a bigger cache.
- Processor performance (CPI) is directly tied to AMAT: fewer stalls from cache misses mean the pipeline is fed more consistently, improving effective Instructions Per Cycle (IPC).
- System throughput benefits because the memory bus and lower-level memory (DRAM) are accessed less frequently, freeing bandwidth for other cores, DMA

transfers, or I/O — this is especially significant in multi-core systems sharing an L3 cache.

- Excessively large caches can also increase static (leakage) power and chip area, which can indirectly reduce achievable clock frequency — an important design trade-off beyond pure timing.

3. Summary Table

Cache Size Change	Hit Rate	Access Time (Hit Time)	AMAT / Latency	Throughput Impact
Small → Medium increase	Increases sharply	Increases slightly	Decreases (net gain)	Improves
Medium → Large increase	Increases marginally	Increases moderately	May flatten or rise	Plateaus
Very large (over-sized)	Near saturation	Increases significantly	Increases (net loss)	Degrades

Conclusion: Cache size must be tuned to the working-set size of typical workloads. Doubling cache capacity does not guarantee proportional performance gains; architects instead balance size against access latency, area, and power to minimize AMAT and maximize sustained throughput.

Question 2

1. Why Bandwidth Matters for Data-Intensive Workloads

Memory bandwidth is the rate (GB/s) at which data can be moved between memory and the processor/accelerator. Unlike latency, which affects how quickly a single request is served, bandwidth determines how much sustained data throughput a system can support — critical for workloads that stream large volumes of data rather than performing a few random accesses.

2. Application-Specific Impact

Artificial Intelligence (AI) / Deep Learning: Training and inference involve moving massive weight matrices and activation tensors between memory and compute units (GPU/TPU cores). These workloads are often memory-bound rather than compute-bound — the arithmetic intensity (FLOPs per byte moved) is low for operations like matrix-vector multiplication and batch normalization. Insufficient bandwidth (e.g., using DDR instead of

HBM) stalls the compute cores, leaving them idle while waiting for data, directly reducing achievable FLOPS and increasing training time.

Scientific Computing: Applications such as computational fluid dynamics, weather modeling, and molecular simulation operate on huge grids/matrices with streaming access patterns (e.g., stencil computations). These have a high volume of data movement relative to computation, so the achievable performance is frequently capped by the 'memory wall' — described by the Roofline Model, where performance is bounded by $\min(\text{Peak Compute}, \text{Bandwidth} \times \text{Arithmetic Intensity})$.

Multimedia Processing: Video encoding/decoding, image processing, and real-time rendering require continuous high-throughput movement of pixel and frame buffer data. Insufficient bandwidth causes frame drops, stutter, and increased latency in real-time systems (e.g., 4K/8K video, VR rendering), since large blocks of data must be fetched and written back every frame cycle.

3. How Insufficient Bandwidth Creates Bottlenecks

- Processor/accelerator cores stall waiting for operands, lowering effective utilization (a GPU may show high 'occupancy' but low actual throughput if bandwidth-starved).
- Queuing delays build up at the memory controller as concurrent requests from multiple cores or threads compete for limited channel bandwidth, increasing effective latency under load.
- Cache thrashing increases: when the working set exceeds cache capacity and bandwidth cannot refill it fast enough, miss penalties compound.
- Energy efficiency drops because idle cycles waiting on memory still consume static power, reducing performance-per-watt — critical in data centers and mobile AI accelerators.
- Scalability suffers: adding more cores does not improve throughput once the shared memory bus is saturated (Amdahl's-law-like ceiling imposed by bandwidth rather than compute).

4. Mitigation Directions (for context)

- Wider/faster memory interfaces: HBM2/HBM3, GDDR6X in place of standard DDR for bandwidth-critical accelerators.

- Increasing cache hierarchy depth and size to reduce the frequency of DRAM accesses.
- Data compression, tensor tiling, and blocking algorithms that improve arithmetic intensity so less data movement is needed per unit of computation.

Conclusion: For AI, scientific computing, and multimedia workloads, bandwidth — not raw clock speed — is frequently the limiting resource. System designers must match memory bandwidth to the data movement demands of the target workload, or processing throughput will be capped well below the theoretical peak compute capability.

Question 3

1. Background

Virtual memory divides address space into fixed-size pages mapped to physical memory frames via page tables, with the Translation Lookaside Buffer (TLB) caching recent translations. Page size is a key design parameter that affects several, often conflicting, performance dimensions.

2. Effect of Increasing Page Size

Page Faults: Larger pages bring in more contiguous data per fault, which can reduce the number of page faults for workloads with good spatial locality (sequential access). However, for workloads with sparse or random access patterns, larger pages waste I/O bandwidth loading unused data, and a single access to a rarely used region still forces loading an entire large page.

Memory Utilization (Internal Fragmentation): Larger pages increase internal fragmentation — the unused space within the last page allocated to a process grows on average as page size increases. Small processes or those with many small allocations waste proportionally more memory. Smaller pages minimize this waste and give finer-grained memory utilization.

Translation Overhead: Larger pages reduce the number of page-table entries needed to map a given address space, which shrinks page-table size and reduces TLB pressure — more of the working set's translations fit in a limited-entry TLB, cutting TLB miss rate and the frequent page-table walks. Smaller pages require many more entries, increasing translation overhead and the frequency of costly multi-level page-table walks on a TLB miss.

System Throughput: The net throughput effect depends on the balance between these forces. For large, contiguous workloads (e.g., scientific arrays, big-data buffers), large/huge pages (e.g., 2MB or 1GB 'huge pages') significantly boost throughput by reducing TLB misses. For workloads with many small, scattered allocations, smaller pages preserve memory utilization and avoid wasted I/O, sustaining throughput without excess fragmentation.

3. Comparative Summary

Factor	Small Page Size	Large Page Size
Page Fault Frequency	Higher for sequential access (frequent faults)	Lower for sequential access; risk of wasted transfer for random access
Internal Fragmentation	Low — fine-grained allocation	High — more unused space per page
Page Table Size	Large (more entries needed)	Small (fewer entries needed)
TLB Miss Rate / Translation Overhead	Higher (working set spans many entries)	Lower (working set fits in fewer TLB entries)
Best Suited Workload	Many small processes, sparse access	Large contiguous datasets, HPC, databases

4. Conclusion

Page size is a classic architectural trade-off: increasing it reduces translation overhead and can lower page-fault counts for sequential workloads, but at the cost of increased internal fragmentation and potential wasted I/O for sparse access patterns. Modern systems often use multiple page sizes (e.g., 4KB standard pages alongside 2MB/1GB huge pages) so that software can select the appropriate size per workload, balancing memory utilization against translation efficiency and maximizing overall throughput.

Question 4

1. Defining Locality

Temporal Locality: The tendency for a program to access the same memory location repeatedly within a short time window (e.g., loop counters, frequently called function

variables). Caches exploit this by retaining recently accessed data close to the processor so repeat accesses are served quickly.

Spatial Locality: The tendency for a program to access memory locations near one another in a short time window (e.g., iterating through array elements, sequential instruction fetch). Caches exploit this by fetching data in blocks/lines (e.g., 64 bytes) rather than single words, so nearby data is pre-loaded on the first access to a line.

2. Influence on Cache Hit Rate

- Strong temporal locality directly raises hit rate because repeated references find data already resident from a prior access, avoiding a trip to lower memory levels.
- Strong spatial locality raises hit rate because a single cache-line fetch satisfies multiple subsequent nearby accesses — this is the principle behind prefetching and block-based cache design.
- Programs with poor locality (e.g., random pointer chasing in linked lists, sparse matrix access) exhibit low hit rates regardless of cache size, since there is little reusable or nearby data to exploit.

3. Influence on Memory Access Latency

Since hits are served from fast on-chip cache (a few cycles) while misses require access to slower levels (L2, L3, or DRAM, costing tens to hundreds of cycles), higher locality directly reduces the observed Average Memory Access Time (AMAT). Each additional hit avoids a miss penalty, so latency scales inversely with the degree of locality present in an access pattern.

4. Influence on Processor Efficiency

- Higher hit rates keep the pipeline supplied with instructions/data, reducing stall cycles and improving effective IPC (Instructions Per Cycle).
- Reduced trips to main memory lower contention on the shared memory bus, benefiting multi-core throughput as well as single-thread efficiency.
- Compiler and algorithm-level optimizations (loop tiling/blocking, array-of-structures vs structure-of-arrays layout, loop interchange) are explicitly designed to increase temporal and spatial locality, directly translating into measurable speedups without changing the underlying hardware.

- Reduced main-memory traffic also lowers dynamic energy consumption, improving performance-per-watt, which is significant for both mobile and data-center processors.

5. Conclusion

Locality is the fundamental principle that makes cache memory effective at all. Temporal locality justifies keeping recently used data on-chip, while spatial locality justifies fetching data in blocks. Together they determine the achievable hit rate, which in turn governs AMAT, pipeline efficiency, and overall processor throughput — making locality-aware software and hardware design a central concern in computer architecture.

Question 5

1. Technology Overview

SRAM (Static RAM): Uses a 6-transistor bistable latch per bit; no refresh required. Fastest available memory technology but has the lowest bit-density (largest cell area), making it expensive per bit.

DRAM (Dynamic RAM): Stores each bit as charge on a capacitor accessed via a single transistor; requires periodic refresh (every few ms) to prevent charge leakage. Offers much higher density than SRAM at lower cost, but is slower and volatile.

NAND Flash: Non-volatile floating-gate/charge-trap technology used for secondary storage (SSDs). Very high density and non-volatility, but significantly slower than DRAM/SRAM and has limited write endurance (wears out after repeated program/erase cycles).

MRAM (Magnetoresistive RAM): Stores bits via magnetic tunnel junctions (spin orientation) rather than charge. Non-volatile, with near-SRAM speed and very high endurance, but currently lower density and higher cost per bit than DRAM.

RRAM (Resistive RAM): Stores bits as resistance states in a metal-oxide layer. Non-volatile, offers good density and scalability potential, with moderate speed and endurance — an emerging candidate for both storage-class memory and embedded non-volatile memory.

2. Comparative Evaluation

Technology	Latency	Bandwidth	Capacity/Density	Energy Use	Endurance	Relative Cost/bit
SRAM	Very low (sub-ns, 1–3 cycles)	Very high	Low (large cell)	Low static, low dynamic per access	Unlimited	Very high
DRAM	Low (tens of ns)	High	High	Moderate (needs refresh power)	Effectively unlimited	Moderate
NAND Flash	High (μ s–ms for writes)	Moderate	Very high	Low idle power (non-volatile)	Limited (10^3 – 10^5 P/E cycles)	Very low
MRAM	Low–moderate (ns range)	Moderate–high	Moderate	Low (no refresh, non-volatile)	Very high (10^{12+} cycles)	High
RRAM	Moderate (tens of ns)	Moderate	High (scalable)	Low (non-volatile)	Moderate–high (10^6 – 10^9 cycles)	Moderate

3. Suitability Across the Memory Hierarchy

Register / L1–L3 Cache: SRAM is the only viable choice due to its speed matching processor clock rates; its high cost and low density are acceptable because cache capacities are small (KB–MB range).

Main Memory (DRAM tier): DRAM remains the standard choice, balancing density, cost, and acceptable latency for gigabyte-scale main memory, despite the overhead of refresh cycles and volatility.

Secondary / Persistent Storage: NAND Flash (in SSDs) dominates due to very high density and low cost per bit combined with non-volatility, suitable for bulk data storage despite slower access and finite write endurance.

Emerging / Storage-Class Memory (gap between DRAM and Flash): MRAM and RRAM are strong candidates for next-generation non-volatile working memory or persistent caches — MRAM suits applications needing very high endurance and near-DRAM speed (e.g., automotive, industrial control, instant-on systems), while RRAM suits high-density embedded non-volatile memory and neuromorphic/in-memory computing applications where scalability matters more than extreme endurance.

4. Application-Based Recommendations

- High-performance CPUs/GPUs (cache hierarchy): SRAM for L1–L3 — speed is non-negotiable at this level.

- General-purpose system main memory: DRAM — best balance of cost, density, and performance for GB-scale capacity.
- Consumer storage / SSDs / mobile devices: NAND Flash — non-volatility and low cost per bit are essential for bulk storage.
- Safety-critical or high-endurance embedded systems (automotive ECUs, IoT sensors requiring frequent writes): MRAM — near-unlimited endurance and non-volatility with fast access.
- Emerging non-volatile main memory, in-memory computing, and dense embedded NVM: RRAM — good scalability and density with acceptable speed for next-generation architectures.

5. Conclusion

No single memory technology satisfies every requirement of a hierarchy simultaneously — a fundamental fact that motivates hierarchical memory design itself. SRAM sacrifices density and cost for speed at the top of the hierarchy; DRAM balances speed, density, and cost for main memory; NAND Flash sacrifices speed for extreme density and non-volatility at the storage tier; and emerging technologies like MRAM and RRAM aim to close the latency-versus-non-volatility gap between DRAM and Flash, each suited to different endurance, density, and application requirements.